

Graph Deep Learning

Fabian Bosshard

February 3, 2026

Contents

1	Spectral theorem	1
2	Graphs	2
2.1	normalized adjacency and multi-hop propagation	2
2.2	graph Laplacian	2
2.3	Cheeger inequality	3
2.4	effective resistance	3
2.4.1	Interpretation	3
2.4.2	Connection to random walks	3
3	Graph neural networks	4
3.1	Graph shift operators	4
3.2	Graph convolutions	4
3.3	Examples of choices for $\tilde{\mathbf{A}}$	4
3.4	Message passing	5
3.4.1	Graph attention networks	5
3.4.2	Edge-conditioned convolution	5
3.5	A good recipe	6
3.6	Over-smoothing	6
4	Pooling on graphs	7
4.1	SRC decomposition	7
4.2	Global pooling	7

1 Spectral theorem

Theorem 1.1. Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be symmetric, i.e. $\mathbf{A}^\top = \mathbf{A}$. Then,

$$\mathbf{A} = \sum_{i=1}^n \lambda_i \mathbf{u}_i \mathbf{u}_i^\top \quad (1.1)$$

with orthonormal eigenvectors $\mathbf{u}_1, \dots, \mathbf{u}_n \in \mathbb{R}^n$ and real eigenvalues $\lambda_1, \dots, \lambda_n \in \mathbb{R}$. Equivalently,

$$\mathbf{A} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top \quad (1.2)$$

with $\mathbf{U} := [\mathbf{u}_1 \cdots \mathbf{u}_n]$, $\mathbf{U}^\top \mathbf{U} = \mathbf{I}_n$ and $\mathbf{\Lambda} := \text{diag}(\lambda_1, \dots, \lambda_n)$. $\triangleleft$

Theorem 1.1 is extensively used in Principal Component Analysis (PCA) to reduce the complexity of the input space (it is applied to the covariance matrix of the inputs).

2 Graphs

Let $G = (V, E)$ be an undirected graph with the adjacency matrix $\underline{A} \in \mathbb{R}^{n \times n}$

$$[\underline{A}]_{uv} = \begin{cases} 1 & (u, v) \in E \\ 0 & (u, v) \notin E \end{cases} \quad (2.1)$$

where $[\cdot]_{uv}$ denotes the entry in row u and column v .

The *diagonal degree matrix* $\underline{D} \in \mathbb{R}^{n \times n}$ is defined by

$$[\underline{D}]_{uv} = \begin{cases} d_u & u = v \\ 0 & u \neq v \end{cases} \quad (2.2)$$

where d_u is the degree of node u , i.e. $\underline{D}$ simply places all node degrees on the diagonal.

2.1 normalized adjacency and multi-hop propagation

Definition 2.1. The *symmetrically normalized adjacency matrix* is

$$\hat{\underline{A}} = \underline{D}^{-1/2} \underline{A} \underline{D}^{-1/2} \quad (2.3)$$

or, entrywise,

$$[\hat{\underline{A}}]_{uv} = \begin{cases} \frac{1}{\sqrt{d_u d_v}} & (u, v) \in E \\ 0 & (u, v) \notin E \end{cases}$$

Fact 2.1 (multi-hop propagation). The entry $[\hat{\underline{A}}^k]_{vu}$ can be computed explicitly as follows:

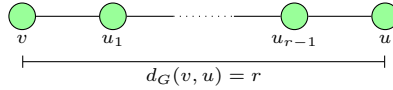
$$[\hat{\underline{A}}^k]_{vu} = \sum_{\pi} \prod_{(x,y) \in E_{\pi}} \frac{1}{\sqrt{d_x d_y}} \quad (2.4)$$

the sum is over all walks $\pi = (v, \dots, u)$ of length k from v to u and the product is over the edges $E_{\pi} = \{(v, u_1), \dots, (u_{k-1}, u)\}$ on the walk. $\triangleleft$

Corollary 2.2. Let $v, u \in V$ with $r = d_G(v, u)$, where $d_G(\cdot, \cdot)$ denotes the shortest-path distance. Assume there is exactly one path

$$(v, u_1, \dots, u_{r-1}, u)$$

of length r between v and u :



Then

$$[\hat{\underline{A}}^r]_{vu} = \frac{1}{\sqrt{d_v d_{u_1}}} \cdot \prod_{i=1}^{r-2} \frac{1}{\sqrt{d_{u_i} d_{u_{i+1}}}} \cdot \frac{1}{\sqrt{d_{u_{r-1}} d_u}} = \frac{1}{\sqrt{d_v d_u}} \prod_{i=1}^{r-1} \frac{1}{d_{u_i}} \quad (2.5)$$

$\triangleleft$

2.2 graph Laplacian

Definition 2.2. The *combinatorial* graph Laplacian is

$$\underline{L} = \underline{D} - \underline{A} \quad (2.6)$$

and the *normalized* graph Laplacian is

$$\hat{\underline{L}} = \underline{D}^{-1/2} \underline{L} \underline{D}^{-1/2} = \underline{D}^{-1/2} (\underline{D} - \underline{A}) \underline{D}^{-1/2} \stackrel{(2.3)}{=} \underline{I}_n - \hat{\underline{A}} \quad (2.7)$$

Both are symmetric and positive semidefinite, and their eigenvalues satisfy

$$0 = \lambda_0 \leq \lambda_1 \leq \dots \leq \lambda_{n-1}$$

λ_1 is called the *spectral gap*. The number of zero eigenvalues (i.e., the multiplicity of the 0 eigenvalue) equals the number of connected components of the graph. $\triangleleft$

To understand Definition 2.2, consider a function $f: V \rightarrow \mathbb{R}$. Denote by $\mathbf{f} \in \mathbb{R}^n$ the vector whose v -th entry is $f(v)$. Then

$$(\hat{\mathbf{L}}\mathbf{f})_v = f(v) - \frac{1}{\sqrt{d_v}} \sum_{(u,v) \in E} \frac{f(u)}{\sqrt{d_u}} \quad (2.8)$$

i.e., $(\hat{\mathbf{L}}\mathbf{f})_v$ is the value at v minus a degree-normalized average of the neighbors. This is why the Laplacian is often viewed as a *discrete second derivative* on the graph: *it measures how much f at v deviates from its neighborhood*.

If we plot the eigenvectors of the Laplacian, it resembles that of a signal.

Another important identity is the quadratic form

$$\mathbf{f}^\top \mathbf{L} \mathbf{f} = \frac{1}{2} \sum_{(u,v) \in E} (f(u) - f(v))^2 \quad (2.9)$$

which shows that $\mathbf{L}$ (and hence also $\hat{\mathbf{L}}$) is positive semidefinite, since the right-hand side is always nonnegative. Moreover, (2.9) is small exactly when f varies slowly across edges, so the Laplacian encodes the *smoothness* of functions on the graph.

2.3 Cheeger inequality

The *Cheeger inequality* relates the spectral gap λ_1 to the *Cheeger constant* $h(G)$, which measures how difficult it is to separate the graph into two large pieces. It states, in particular, that

$$\frac{1}{2}h(G)^2 \leq \lambda_1 \leq 2h(G),$$

so a larger spectral gap implies that the graph is more “well-connected”.

2.4 effective resistance

Definition 2.3 (effective resistance). View each edge $(u, v) \in E$ as an electrical resistor of resistance 1Ω . The resulting network has a well-defined resistance between any two nodes.

For two nodes $s, t \in V$, the *effective resistance* $R(s, t)$ is defined as the voltage difference needed to send one unit of electrical current from s to t . It can be computed as

$$R(s, t) = (\mathbf{e}_s - \mathbf{e}_t)^\top \mathbf{L}^\dagger (\mathbf{e}_s - \mathbf{e}_t) \quad (2.10)$$

where $\mathbf{L}^\dagger$ is the Moore–Penrose pseudoinverse of the graph Laplacian (2.6) and $\mathbf{e}_v$ is the standard basis vector of vertex v . ◀

2.4.1 Interpretation

If the graph offers many short, parallel paths between s and t , then current can flow easily, so $R(s, t)$ is small. If there are few or long paths, the current is “bottlenecked” and $R(s, t)$ is large. Thus, effective resistance measures how “well-connected” two nodes are inside the global geometry of the graph.

2.4.2 Connection to random walks

A *random walk* on G is the Markov chain that, from a node v , moves to a uniformly random neighbor of v . Its transition matrix is

$$\mathbf{P} = \mathbf{D}^{-1} \mathbf{A} \quad (2.11)$$

so $\mathbf{P}_{vu} = 1/d_v$ if $(v, u) \in E$. The matrix (2.11) is often called *random-walk matrix*.

For two nodes u, v , the *commute time* $\text{CT}(u, v)$ is the expected number of steps for the random walk to start at u , reach v , and return to u again. It can be related to the effective resistance via

$$\text{CT}(u, v) = 2|E|R(u, v) \quad (2.12)$$

giving a geometric interpretation of how “far apart” two nodes are in terms of random-walk behavior, i.e. two nodes have small commute time exactly when they have small effective resistance.

3 Graph neural networks

3.1 Graph shift operators

Definition 3.1. A matrix $\tilde{\mathbf{A}} \in \mathbb{R}^{n \times n}$ is called a **graph shift operator** (GSO) if it satisfies

$$\tilde{a}_{ij} = 0 \quad \text{whenever } (i, j) \notin E \text{ and } i \neq j$$

where $\tilde{a}_{ij} = [\tilde{\mathbf{A}}]_{ij}$. This means that applying $\tilde{\mathbf{A}}$ to node attributes only mixes information from direct neighbors. Typical choices include the Laplacian (2.6) and the random-walk matrix (2.11). ◀

Applying $\tilde{\mathbf{A}}$ to node attributes $\mathbf{X} \in \mathbb{R}^{n \times d_x}$ is local in the sense that the i -th row of $\tilde{\mathbf{A}}\mathbf{X}$ depends only the neighbors $N(i)$ together with possibly i itself:

$$\mathbf{x}'_i = [\tilde{\mathbf{A}}\mathbf{X}]_{i,:} = \sum_{j=1}^n \tilde{a}_{ij} \mathbf{x}_j = \tilde{a}_{ii} \mathbf{x}_i + \sum_{j \in N(i)} \tilde{a}_{ij} \mathbf{x}_j$$

Using a parameter matrix $\mathbf{\Theta} \in \mathbb{R}^{d_x \times d_h}$, we can apply the filter on a different space:

$$\mathbf{h}_i = [\tilde{\mathbf{A}}\mathbf{X}\mathbf{\Theta}]_{i,:} = \sum_{j=1}^n \tilde{a}_{ij} \mathbf{x}_j \mathbf{\Theta} = \tilde{a}_{ii} \mathbf{x}_i \mathbf{\Theta} + \sum_{j \in N(i)} \tilde{a}_{ij} \mathbf{x}_j \mathbf{\Theta}$$

3.2 Graph convolutions

Let $\mathbf{X} \in \mathbb{R}^{n \times d_x}$ be node features, let $\tilde{\mathbf{A}} \in \mathbb{R}^{n \times n}$ be a GSO and let $\mathbf{\Theta} \in \mathbb{R}^{d_x \times d_h}$ be trainable weights. A **linear graph convolution** is

$$\mathbf{H} = \tilde{\mathbf{A}}\mathbf{X}\mathbf{\Theta} \quad (3.1)$$

where $\mathbf{H} \in \mathbb{R}^{n \times d_h}$ are the transformed node features.

Adding a nonlinearity σ yields a **graph convolutional layer**

$$\mathbf{H} = \sigma(\tilde{\mathbf{A}}\mathbf{X}\mathbf{\Theta}) \quad (3.2)$$

so the parameters can be learned by gradient-based optimization.

Remark 3.1 (multi-hop aggregation). Stacking K layers increases the receptive field. Ignoring nonlinearities for intuition, applying two layers gives $\tilde{\mathbf{A}}(\tilde{\mathbf{A}}\mathbf{X}) = \tilde{\mathbf{A}}^2\mathbf{X}$ which aggregates information from 2-hop neighborhoods. ◀

Two common ways to aggregate up to K hops are **polynomial filters**

$$\mathbf{H}^{(K)} = \sum_{k=0}^K \tilde{\mathbf{A}}^k \mathbf{X} \mathbf{\Theta}^{(k)} \quad (3.3)$$

or a sequence of first-order steps $\mathbf{H}^{(0)} = \mathbf{X}$, $\mathbf{H}^{(k)} = \tilde{\mathbf{A}}\mathbf{H}^{(k-1)}\mathbf{\Theta}^{(k)}$ where nonlinearities can be inserted between layers.

3.3 Examples of choices for $\tilde{\mathbf{A}}$

Several popular layers differ essentially by the choice of $\tilde{\mathbf{A}}$. A standard example is the **GCN normalization**

$$\tilde{\mathbf{A}} = \mathbf{D}^{-1/2}(\mathbf{I}_n + \mathbf{A})\mathbf{D}^{-1/2} \quad (3.4)$$

which **includes self-loops** through $\mathbf{I}_n + \mathbf{A}$. Another example is the random-walk normalization (2.11) which corresponds to averaging over neighbors with probabilities. A third example is the **GIN** choice

$$\tilde{\mathbf{A}} = \mathbf{A} + (1 + \varepsilon)\mathbf{I}_n \quad (3.5)$$

which strengthens the contribution of the root node via ε .

3.4 Message passing

Definition 3.2. Let $\mathbf{x}_i \in \mathbb{R}^{d_x}$ be the feature of node i and let $\mathbf{e}_{ji} \in \mathbb{R}^{d_e}$ be the feature of edge (j, i) . A *message-passing* (MP) layer has the form

$$\mathbf{h}_i = \gamma\left(\mathbf{x}_i, \text{agg}_{j \in N(i)}\{\phi(\mathbf{x}_i, \mathbf{x}_j, \mathbf{e}_{ji})\}\right) \quad (3.6)$$

where

- ϕ is a *message function*, depending on $\mathbf{x}_i$, $\mathbf{x}_j$ and possibly edge features $\mathbf{e}_{ji}$
- agg is a permutation-invariant *aggregation function* (e.g. sum, mean, max)
- γ is an *update function* to obtain new features from aggregated messages and previous features

Note that ϕ and γ are often parametric (e.g. MLPs). ◀

Remark 3.2. Definition 3.2 is the most general (and expressive) form of GNN, encompassing also graph convolutions (3.2) as a special case. (3.2) can be rewritten as

$$\mathbf{h}_i = \sigma\left(\sum_{j \in N(i)} a_{ij} \mathbf{x}_j \underline{\Theta}\right)$$

with $a_{ij} = [\tilde{\mathbf{A}}]_{ij}$. So it fits into Definition 3.2 with $\phi(\mathbf{x}_i, \mathbf{x}_j, \mathbf{e}_{ji}) = a_{ij} \mathbf{x}_j \underline{\Theta}$ (independent of $\mathbf{x}_i$ and $\mathbf{e}_{ji}$), $\text{agg} = \text{sum}$, and $\gamma(\mathbf{x}_i, \cdot) = \sigma(\cdot)$ (independent of $\mathbf{x}_i$). ◀

Message passing operations whose message function ϕ depends only on the sender node's features are called *isotropic*. They are called *anisotropic* when also edge's or receiver node's features are exploited, i.e. ϕ also depends on $\mathbf{e}_{ji}$ or $\mathbf{x}_i$.

3.4.1 Graph attention networks

are a typical example of anisotropic message passing.

1. Transform node features:

$$\mathbf{x}'_i = \mathbf{x}_i \underline{\Theta}_1 \quad (3.7)$$

with $\underline{\Theta}_1 \in \mathbb{R}^{d_x \times d_h}$.

2. Compute attention scores between neighbors:

- 2.1. Score ($\underline{\theta}_2 \in \mathbb{R}^{2d_h}$):

$$\alpha_{ij} = \sigma([\mathbf{x}'_i \parallel \mathbf{x}'_j] \underline{\theta}_2) \quad (3.8)$$

- 2.2. Normalize with softmax over $N(i)$:

$$\tilde{\alpha}_{ij} = \frac{\exp(\alpha_{ij})}{\sum_{k \in N(i)} \exp(\alpha_{ik})} \quad (3.9)$$

3. Aggregate (weighted sum) using attention coefficients as weights:

$$\mathbf{h}_i = \sum_{j \in N(i)} \tilde{\alpha}_{ij} \mathbf{x}'_j \quad (3.10)$$

3.4.2 Edge-conditioned convolution

To incorporate edge attributes into the messages, one may use an MLP $\rho: \mathbb{R}^{d_e} \rightarrow \mathbb{R}^{d_x \times d_h}$ to generate edge-dependent weights. For each edge (j, i) we compute

$$\underline{\Theta}_{ji} = \rho(\mathbf{e}_{ji}) \in \mathbb{R}^{d_x \times d_h} \quad (3.11)$$

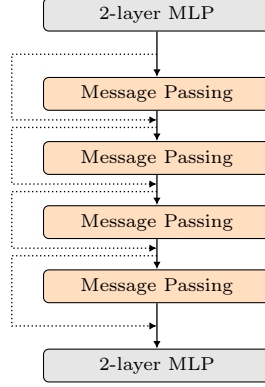
and update nodes by

$$\mathbf{h}_i = \mathbf{x}_i \underline{\Theta}_i + \sum_{j \in N(i)} \mathbf{x}_j \underline{\Theta}_{ji} \quad (3.12)$$

so edges directly control how neighbor information is transformed.

3.5 A good recipe

is to pre- and post-process node features with a 2-layer MLP and to use 4 to 6 message-passing steps in between:



Message passing at the ℓ -th layer:

1. Message:

$$\mathbf{m}_{ji}^\ell = \text{PReLU}\left(\text{BatchNorm}(\mathbf{h}_j^\ell \underline{\Theta}^\ell + \mathbf{b}^\ell)\right) \quad (3.13)$$

2. Aggregate by summation:

$$\mathbf{m}_i^\ell = \sum_{j \in N(i)} \mathbf{m}_{ji}^\ell \quad (3.14)$$

3. Update by concatenation:

$$\mathbf{h}_i^{\ell+1} = \mathbf{h}_i^\ell \parallel \mathbf{m}_i^\ell \quad (3.15)$$

Remark 3.3. This message-passing instance is *isotropic*, since the message (3.13) depends only on the sender embedding $\mathbf{h}_j^\ell$ and not on edge attributes or the receiver features. ◀

3.6 Over-smoothing

Repeated graph convolutions tend to reduce feature differences across neighbors, behaving like low-pass filtering on the graph. After many layers, node representations can become almost indistinguishable within connected components, which can harm node-level prediction tasks.

4 Pooling on graphs

Pooling builds coarser graphs to reduce size or to obtain hierarchical representations.

4.1 SRC decomposition

1. **Selection:** A selection operator computes K supernodes

$$\text{SEL: } G \mapsto S = \{S_1, \dots, S_K\}$$

where each S_k is a set of nodes equipped with nonnegative scores. Equivalently, selection can be encoded by a matrix $\underline{S} \in \mathbb{R}^{K \times n}$.

2. **Reduction:** Given a selection matrix $\underline{S} \in \mathbb{R}^{K \times n}$ and node features $\underline{X} \in \mathbb{R}^{n \times d_x}$, a typical reduction is the weighted aggregation

$$\underline{X}' = \underline{S}\underline{X}$$

which produces pooled features $\underline{X}' \in \mathbb{R}^{K \times d_x}$.

3. **Connection:** A common connection rule builds the pooled adjacency by aggregating edges between supernodes. Using the same $\underline{S}$, a standard choice is

$$\underline{A}' = \underline{S}\underline{A}\underline{S}^\top$$

which yields $\underline{A}' \in \mathbb{R}^{K \times K}$.

Remark 4.1 (spectral intuition). Low-frequency eigenvectors of the Laplacian reveal coarse clusters. A classical pipeline performs clustering (e.g. k -means) in the space spanned by the first few Laplacian eigenvectors, but this can be expensive and ignores attributes. ◀

4.2 Global pooling

For graph-level tasks, one often needs a graph-to-vector map. A *global pooling* (or *readout*) aggregates node embeddings $\{\mathbf{h}_i\}_{i \in V}$ into a single vector and must be permutation-invariant. Typical choices include sum, mean or max pooling, as well as attention-weighted sums.